



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



TFG del Grado en Ingeniería
Informática

VRLearnTobot Laboratorio
Virtual de programación y
robótica



Presentado por Luis Ignacio de Luna Gómez
en Universidad de Burgos — 31 de mayo
de 2025

Tutor: Carlos López Nozal



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



D. Carlos López Nozal, profesor del departamento de nombre departamento, área de nombre área.

Expone:

Que el alumno D. Luis Ignacio de Luna Gómez, con DNI 34809179J, ha realizado el Trabajo final de Grado en Ingeniería Informática titulado título de TFG.

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección del que suscribe, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, 31 de mayo de 2025

Resumen

En este primer apartado se hace una **breve** presentación del tema que se aborda en el proyecto.

Descriptores

Palabras separadas por comas que identifiquen el contenido del proyecto Ej: servidor web, buscador de vuelos, android ...

Abstract

A **brief** presentation of the topic addressed in the project.

Keywords

keywords separated by commas.

Índice general

Índice general	iii
Índice de figuras	v
Índice de tablas	vi
1. Introducción	1
2. Objetivos del proyecto	5
2.1. Introducción	5
2.2. Objetivos Funcionales	6
2.3. Objetivos No Funcionales	7
3. Conceptos teóricos	9
3.1. Plataformas de Programación Visual y Kits de Robótica	9
3.2. Arquitectura y Componentes del Prototipo en Unity	11
3.3. Estructura Lógica de los Bloques (BlockModel)	13
3.4. Conexiones (ConnectionModel)	14
3.5. Entradas (InputModel)	16
3.6. Campos (FieldModel)	16
3.7. Variables (VariableMap)	17
3.8. Sistema de Ejecución (CSharpRunner e Cmdtors)	17
3.9. Representación Visual y Composición de Bloques en Unity	18
3.10. Serialización y Persistencia	22
3.11. Objetivos de Desarrollo Sostenible (ODS) y su Aplicación en el Proyecto	22

4. Técnicas y herramientas	25
4.1. Metodología de Desarrollo: Agile con SCRUM	25
4.2. Herramientas Core para el Desarrollo de Software	26
4.3. Persistencia de Datos del Programa	28
4.4. Herramientas para Gestión de Proyectos y Desarrollo	29
5. Aspectos relevantes del desarrollo del proyecto	31
5.1. Metodología de Desarrollo: Iteraciones Ágiles con SCRUM	31
5.2. Decisiones de Diseño Arquitectónico e Implementación	32
5.3. Detalles Clave de Implementación	34
5.4. Retos Técnicos Abordados y Estrategias de Depuración	38
6. Trabajos relacionados	41
6.1. Plataformas de Programación por Bloques	41
6.2. Simuladores y Entornos de Robótica Educativa	42
6.3. Herramientas para el Desarrollo de Interfaces de Bloques	43
6.4. Síntesis de la Contribución del Proyecto	44
7. Conclusiones y Líneas de trabajo futuras	45
7.1. Conclusiones	45
7.2. Líneas de Trabajo Futuras	46
7.3. Conclusión Final	50
Bibliografía	51

Índice de figuras

3.1. Estructura del Prefab <code>motion_movesteps</code> en la Jerarquía de Unity.	19
--	----

Índice de tablas

3.1. Comparación de Scratch y Blockly	10
3.2. Comparación de LEGO WeDo 2.0 y LEGO SPIKE Prime . . .	10

1. Introducción

En la última década, el interés por la enseñanza de la programación y la robótica ha crecido de manera significativa, con especial énfasis en la educación preuniversitaria, dirigida a **niños y adolescentes**. Esto se debe a la creciente necesidad de dotar a las nuevas generaciones de habilidades fundamentales en pensamiento computacional, resolución de problemas y lógica algorítmica, consideradas esenciales para su desarrollo en el mundo digital actual [21]. La alfabetización digital y el pensamiento computacional son habilidades transversales cruciales que promueven la resolución creativa de problemas y preparan a los estudiantes para los desafíos de la sociedad moderna [11, 3].

Entre las metodologías más empleadas en la enseñanza de la programación, la **programación basada en bloques** ha demostrado ser una herramienta pedagógica eficaz. Este paradigma visual facilita la comprensión de la lógica de programación y el flujo de control sin la complejidad de la sintaxis textual, reduciendo la barrera de entrada para estudiantes novatos y promoviendo un aprendizaje más accesible y lúdico [22, 14]. Su efectividad se ha demostrado en diversos contextos educativos, como la incorporación de robótica en el currículo STEM a través de herramientas de bloques como Scratch [7]. Experiencias en la introducción a la programación en la universidad han mostrado que el uso de estos lenguajes ha mejorado significativamente las tasas de aprobación en asignaturas fundamentales de la informática [14].

Actualmente, dos de las plataformas más destacadas en este ámbito son **Scratch y Blockly**. Scratch, desarrollado por el MIT Media Lab, se ha consolidado como una de las herramientas educativas más utilizadas globalmente para la enseñanza de la programación, proporcionando un entorno altamente accesible y motivador [22]. Por otro lado, Blockly, desarrollado por Google,

ofrece una biblioteca de código abierto que permite integrar programación por bloques en diversas plataformas y puede convertir los bloques visuales a lenguajes de programación textuales como JavaScript y Python, ofreciendo así una mayor versatilidad en la generación de código y aplicaciones [14]. Aunque **Scratch utiliza una implementación personalizada basada en los principios de Blockly**, existen diferencias fundamentales entre ambos sistemas, especialmente en términos de flexibilidad y aplicación práctica [2]. La adopción y los avances en este campo se observan con gran desarrollo en diversas regiones de América Latina, donde la robótica educativa se ha posicionado como un medio eficaz para la enseñanza de habilidades STEM en docentes de primaria, evidenciando un compromiso temprano con estas metodologías [19].

La **gamificación** ha emergido como un enfoque clave en la enseñanza de la programación. Al introducir elementos propios de los juegos en contextos no lúdicos, se promueve una motivación intrínseca y un compromiso activo del estudiante en el proceso de aprendizaje, logrando así una experiencia más efectiva y significativa [2]. Estudios recientes confirman que los proyectos de robótica aumentan significativamente la motivación y el engagement de los estudiantes, facilitando un aprendizaje más profundo y aplicado [6]. Además, la integración de la gamificación con la programación basada en bloques (incluso incluyendo conceptos de inteligencia artificial) demuestra resultados positivos en el rendimiento académico y la motivación en estudiantes de primaria y secundaria [1]. Es en este cruce entre la rigurosa aplicación de los conocimientos técnicos y la creatividad del diseño, donde reside la oportunidad de desarrollar herramientas capaces de **vincular y emocionar a los estudiantes**, transformando el aprendizaje de la programación en una actividad lúdica y accesible que genere un impacto duradero.

No obstante, a pesar de estos avances metodológicos y didácticos, existen desafíos significativos para la enseñanza de la programación y robótica, especialmente en entornos con acceso limitado a recursos educativos y tecnológicos. La falta de laboratorios físicos y la dependencia de materiales y robots costosos restringen la accesibilidad de estas valiosas herramientas, afectando principalmente a estudiantes en zonas rurales o con restricciones económicas [21]. La inviabilidad de replicar laboratorios con equipamiento sofisticado en contextos de bajo presupuesto y la dificultad de acceder a prácticas con componentes reales impulsan la búsqueda de alternativas flexibles y accesibles [21]. Esta problemática está directamente alineada con los **Objetivos de Desarrollo Sostenible (ODS)** de la Organización de las Naciones Unidas (ONU), en particular el **ODS 4 (Educación de calidad)**, que busca garantizar una educación inclusiva, equitativa y de

calidad para niños y adolescentes [20]. Se enfoca en que nadie se quede atrás en el acceso a oportunidades educativas, y el **ODS 10 (Reducción de las desigualdades)**, que promueve la reducción de las brechas sociales y económicas entre regiones y poblaciones, buscando asegurar un acceso equitativo al conocimiento y a las oportunidades educativas [20].

Frente a esta problemática, los **entornos de simulación virtual y los laboratorios virtuales de robótica** han demostrado ser una solución excepcionalmente eficaz para *democratizar el acceso* a la enseñanza de la programación y robótica. Estas plataformas permiten a los estudiantes diseñar, construir y programar robots virtuales en un entorno digital, eliminando la necesidad de hardware físico costoso y ofreciendo un espacio de experimentación seguro y flexible. Permiten "proporcionar una experiencia similar a la obtenida en un laboratorio de prácticas.^a a través de sistemas de apoyo con gran realismo [15]. El uso de **juegos serios** (serious games) y simulaciones interactivas se ha consolidado como una metodología prometedora para la enseñanza de la programación, al integrar desafíos divertidos y elementos educativos que mejoran la retención del conocimiento e incrementan significativamente la motivación y la participación activa de los estudiantes [18].

Si bien el desarrollo de tecnologías y la investigación en áreas individuales como la programación por bloques, la simulación 3D y la gamificación es sólida, la *integración holística y accesible* de estos elementos en un *único sistema orientado a la robótica educativa específica de bajo coste* sigue siendo un área con *oportunidades significativas de desarrollo e implementación innovadora*. Pocas soluciones existentes logran un balance óptimo entre una interfaz amigable tipo Scratch, la potencia de un motor 3D y la flexibilidad para ser desplegadas en plataformas móviles de forma sencilla y gratuita. Este proyecto busca precisamente aportar valor en este nicho.

En este contexto, el presente **Trabajo Fin de Grado** se orienta al desarrollo de un **laboratorio virtual de programación y robótica**. La plataforma está diseñada en el motor de videojuegos **Unity** utilizando **C#**, con el objetivo de ofrecer una alternativa *accesible, interactiva y didáctica* para la metodología **STEAM**. La herramienta permitirá a sus usuarios aprender conceptos básicos de programación y robótica interaccionando con bloques de programación visual de **Scratch** para simular el comportamiento de un robot **LEGO WeDo 2.0** en un entorno 3D, priorizando su ejecución en **dispositivos Android tipo tablet**.

Aunque la literatura presenta diversas plataformas, la *integración de un sistema de programación visual completo* (como Blockly o Scratch) *con una simulación 3D interactiva de un robot educativo concreto en un motor de juegos como Unity*, que además sea accesible en plataformas móviles, aún presenta un área con *oportunidades de desarrollo* significativas [7, 10]. Este proyecto busca aportar valor en este nicho. Dado que Blockly no está completamente adaptado a la estructura **visual y de interacción** de Scratch, este proyecto no plantea una simple integración. En su lugar, se propone el desarrollo de una **implementación híbrida** que combine las ventajas de la filosofía lúdica de Scratch con la versatilidad técnica de una biblioteca abierta. Para ello, se analizará y adaptará la implementación de **UBlockly**, una librería de programación por bloques en Unity, que si bien no está diseñada específicamente para replicar los bloques de Scratch, se ha identificado como una referencia sólida y un punto de partida técnico esencial para la adaptación del sistema propuesto [9].

Este proyecto también plantea un desafío técnico importante: la integración **fluida** de un entorno de programación visual **complejo** en un motor de simulación en 3D **interactivo**. Para abordar este desafío, se explorarán estrategias utilizadas en otras plataformas **relacionadas con la robótica educativa y el pensamiento computacional** [5], identificando sus fortalezas y limitaciones para informar el diseño de una solución mejorada y optimizada. Asimismo, se analizará el impacto de la gamificación y la accesibilidad en el aprendizaje, con el objetivo de diseñar un entorno intuitivo y motivador que pueda ser utilizado por estudiantes con distintos niveles de experiencia **y recursos**.

2. Objetivos del proyecto

2.1. Introducción

El presente Trabajo Fin de Grado tiene como objetivo principal el desarrollo de un **prototipo de laboratorio virtual de programación y robótica** que permita a los estudiantes aprender programación mediante bloques. Este laboratorio ha sido desarrollado en el motor de videojuegos **Unity 6** y utilizando el lenguaje de programación **CSharp**.

La educación en robótica y programación ha demostrado ser una herramienta clave en el aprendizaje del pensamiento computacional. Sin embargo, uno de los principales desafíos en la enseñanza de la robótica es la dependencia de hardware físico, lo que limita el acceso a estudiantes en contextos de escasos recursos.

Para abordar esta problemática, el laboratorio virtual combinará un **entorno de simulación 3D**, que permitirá visualizar la ejecución de los programas, con un **sistema de bloques de programación**, basado en la lógica de **Scratch**. Este sistema se inspira en la robótica educativa, con la intención de simular a futuro un robot tipo LEGO WeDo 2.0 virtual, implementándose en este prototipo los mecanismos de movimiento fundamentales.

Además, se ha analizado el repositorio **UBlockly**, una implementación en Unity que permite la programación por bloques. Aunque UBlockly no está diseñado específicamente para replicar los bloques de Scratch, ha representado una base sólida y clave para evaluar su adaptabilidad e integración en este trabajo [9]. Su estudio ha permitido definir estrategias para la implementación y optimización del sistema propuesto en este proyecto.

Para ello, se establecen los siguientes objetivos específicos que se han perseguido y alcanzado, parcial o totalmente, durante el desarrollo del prototipo:

2.2. Objetivos Funcionales

Estos objetivos están relacionados con las funcionalidades que el prototipo de laboratorio virtual debe cumplir para ofrecer una experiencia educativa inicial.

1. Diseñar e implementar el núcleo de un sistema de programación por bloques.

- Replicar la lógica de interacción visual y el comportamiento fundamental de Scratch (arrastre, conexión y ejecución de bloques).
- Adaptar e integrar la estructura de UBlockly como base para la gestión de bloques y su traducción a acciones ejecutables en Unity. Esto ha implicado el diseño e implementación de los siguientes bloques *funcionales en el prototipo*:
 - **Bloque de movimiento:** (`motion_movesteps`) para controlar el desplazamiento lineal del robot en la simulación 3D.
 - **Bloque de eventos:** (`event_whenflagclicked`) para iniciar la secuencia de ejecución del programa.
- Evaluar y aplicar aspectos clave de UBlockly para lograr una implementación eficiente y escalable de la lógica de programación visual tipo Scratch en Unity.

2. Integrar el sistema de bloques con un entorno de simulación 3D en Unity.

- Implementar un entorno de programación visual (`WorkspaceView` y `BlockListView`) como una interfaz interactiva en Unity donde se pueden arrastrar, soltar y conectar bloques replicando el entorno Scratch.
- Desarrollar un sistema de ejecución de bloques (`CSharpRunner` y `Cmdtors`) que traduzca dinámicamente las instrucciones programadas con bloques en acciones ejecutadas por el robot simulado en Unity.

- Configurar un entorno de simulación 3D (`RobotBehaviour` y escenas) que permita la visualización en tiempo real del comportamiento del robot virtual (`RobotBehaviour`) al ejecutar las instrucciones de los bloques.
- Diseñar una interfaz de usuario intuitiva (`UICanvasView`) que facilite la transición entre los modos de programación y simulación, orientada a la accesibilidad y facilidad de uso para el público objetivo.

3. Implementar un prototipo de simulación para un robot LEGO WeDo 2.0.

- Crear un modelo 3D básico de un robot genérico inspirado en la estética LEGO WeDo 2.0.
- Dotar al robot virtual de funcionalidades esenciales para su movimiento en el entorno simulado, ejecutando comandos interpretados desde los bloques (ej. desplazamiento).
- Asegurar que la experiencia de simulación demuestre el flujo completo desde la interfaz de bloques hasta la acción visual del robot en el entorno 3D, validando la viabilidad del concepto principal.

2.3. Objetivos No Funcionales

Estos objetivos se enfocan en las cualidades transversales de la solución, como su accesibilidad, rendimiento y usabilidad.

1. Accesibilidad y escalabilidad del prototipo.

- Desarrollar una plataforma que pueda ejecutarse en dispositivos de hardware común y sin requerimientos técnicos elevados, con el objetivo de priorizar la portabilidad a plataformas Android (Tablets).
- Explorar estrategias de diseño y desarrollo que minimicen la dependencia de equipamiento costoso, haciendo la herramienta accesible a estudiantes en zonas con recursos limitados.

2. Investigación y análisis.

- Investigar y analizar plataformas existentes de programación y simulación robótica (ej. CoSpaces Edu, Robot Virtual Worlds, VEXcode VR y CoderZ) para comprender sus fortalezas y limitaciones.
- Definir qué aspectos de las soluciones existentes pueden ser mejorados o adaptados en la solución propuesta de este TFG, sentando las bases para futuras evoluciones del sistema.

3. Adquisición de conocimientos técnicos.

- Adquirir conocimientos y habilidades en el uso del motor Unity 6 y el lenguaje C# para la creación de entornos interactivos y simulaciones educativas.
- Aplicar buenas prácticas en el diseño de software y en la integración de sistemas de programación visual dentro de un motor de simulación, incluyendo la aplicación de patrones de diseño (MVC, Observador, Fábrica) y la gestión eficiente de estructuras de datos.

3. Conceptos teóricos

La implementación del laboratorio virtual desarrollado en este Trabajo Fin de Grado se fundamenta en una evaluación previa de diversas plataformas de programación visual. Este capítulo detalla los conceptos teóricos esenciales que sustentan el prototipo, con especial atención a la arquitectura de la librería UBlockly y su adaptación para este proyecto.

3.1. Plataformas de Programación Visual y Kits de Robótica

Comparación entre Scratch y Blockly

Actualmente, Scratch y Blockly son las plataformas dominantes en el ámbito de la programación visual basada en bloques para la educación. Su análisis comparativo ha sido fundamental para definir la dirección del presente proyecto, como se detalla en la Tabla 3.1:

Comparación entre LEGO WeDo 2.0 y LEGO SPIKE Prime

La elección de LEGO WeDo 2.0 como el robot de referencia para la simulación en este proyecto se justifica al analizar sus diferencias fundamentales con otras plataformas de robótica educativa de LEGO, como se detalla en la Tabla 3.2:

LEGO SPIKE Prime es una adaptación de Scratch hecha por LEGO, por lo que mantiene una estructura de bloques similar a Scratch, pero ajustada a su ecosistema de hardware. LEGO WeDo 2.0 usa una interfaz más sencilla

Característica	Scratch	Blockly
Paleta de colores	Fija (según categorías)	Personalizable
Categorización de bloques	Movimiento, Control, Sensores, Eventos, Operadores, Variables, Funciones	Modular y personalizable
Exportación de código	No permite exportar código	Genera código en múltiples lenguajes (JavaScript, Python, Lua)
Flexibilidad	Cerrado, no permite modificar bloques	Altamente personalizable
Uso principal	Aprendizaje de programación visual en educación	Desarrollo de interfaces de programación visual para integración con código real

Tabla 3.1: Comparación de Scratch y Blockly

Característica	LEGO WeDo 2.0	LEGO SPIKE Prime
Paleta de colores	Basada en íconos	Adaptación de Scratch, con bloques en formato palabra e íconos
Categorización de bloques	Basada en hardware (motores, sensores)	Similar a Scratch, con bloques específicos para hardware LEGO
Tipo de bloques	Bloques verticales con íconos sin palabras	Bloques de palabras en horizontal y algunos bloques con íconos
Exportación de código	No permite exportar código	No permite exportar código
Flexibilidad	Cerrado, bloques predefinidos	Más flexible que WeDo 2.0, pero limitado al ecosistema LEGO
Uso principal	Programación básica para principiantes en robótica educativa	Programación avanzada para proyectos de robótica con mayor capacidad de personalización

Tabla 3.2: Comparación de LEGO WeDo 2.0 y LEGO SPIKE Prime

con bloques puramente icónicos (sin palabras), mientras que SPIKE Prime combina texto e íconos.

3.2. Arquitectura y Componentes del Prototipo en Unity

La implementación del laboratorio virtual en Unity se basa en la adaptación de la librería UBlockly, que proporciona un marco robusto para la programación visual. UBlockly, al estar diseñado sobre los principios de Blockly, ofrece una estructura modular y personalizable que ha sido fundamental como punto de partida para replicar la experiencia de Scratch en un entorno 3D. Este apartado describe la arquitectura principal y los componentes fundamentales que han sido desarrollados o adaptados en el prototipo.

Arquitectura General (Modelo-Vista-Controlador - MVC)

El prototipo adopta una arquitectura de software basada en el patrón Modelo-Vista-Controlador (MVC) para promover la modularidad, la mantenibilidad y la escalabilidad del sistema. Esta separación de intereses es crucial para gestionar la complejidad de un entorno interactivo y una simulación 3D.

- **Modelo (Data & Logic):** Representa la lógica de negocio y el estado del programa. Incluye las estructuras de datos que definen los bloques (`BlockModel`), sus conexiones (`ConnectionModel`), entradas (`InputModel`), campos (`FieldModel`) y la organización del espacio de trabajo (`WorkSpaceModel`), así como la gestión de variables (`VariableMap`) y procedimientos (`ProcedureDB`). Las operaciones de datos se abstraen de la presentación visual.
- **Vista (UI & Visuals):** Se encarga de la representación visual del Modelo. Incluye todos los elementos de la interfaz de usuario de Unity (`UnityEngine.UI.Image`, `TMPPro.TextMeshProUGUI`, `UnityEngine.UI.Button`) y la representación 3D del robot (`RobotBehaviour`). Las vistas observan los cambios en el modelo y se actualizan automáticamente sin modificar directamente los datos subyacentes.

- **Controlador (Orquestación & Interaction Logic):** Maneja la interacción del usuario y la lógica de la aplicación, coordinando las comunicaciones entre Modelos y Vistas. Clases como `AppController`, `WorkspaceController`, `BlockDragController`, `BlockConnectionController` e `ExecutionController` operan en esta capa, traduciendo las entradas del usuario y las acciones del programa en manipulaciones del Modelo o actualizaciones de la Vista.

Espacio de Trabajo (`WorkspaceModel`)

El `WorkspaceModel` es el contenedor principal que gestiona el estado lógico y la organización de todos los elementos del programa de bloques. Su principal función es mantener la persistencia y disposición de los bloques, variables y procedimientos en el entorno de programación.

- **Atributos principales del Workspace:**
 - **Id:** Identificador único para el espacio de trabajo.
 - **Options:** Configuraciones que definen el comportamiento global, como el modo de solo lectura o la ejecución sincrónica.
 - **TopBlocks:** `List<BlockModel>` de todos los bloques que no están conectados a otros bloques en una pila superior (es decir, bloques en la cima de la jerarquía).
 - **BlockDB:** `Dictionary<string, BlockModel>` que almacena todos los bloques presentes en el espacio de trabajo, indexados por su ID único.
 - **VariableMap:** Estructura (`VariableMap`) para la gestión de variables de usuario, incluyendo su creación, eliminación y renombrado.
 - **ConnectionDBList:** `Dictionary<EConnection, BlockConnectionDB>` que organiza y gestiona las bases de datos de conexiones, optimizando la búsqueda y validación de uniones entre bloques.
 - **ProcedureDB:** Base de datos (`ProcedureDB`) que gestiona los procedimientos o funciones definidos por el usuario dentro del espacio de trabajo.

Ciclo de Vida de Bloques: Creación y Eliminación

En UBlockly, la gestión de la vida de los bloques sigue un ciclo controlado. La `BlockFactory` se encarga de instanciar un `BlockModel` y lo

inicializa con su estructura lógica (*BlockDefinition*), incluyendo sus entradas, campos y conexiones. Este proceso garantiza que cada bloque se configure correctamente desde el momento de su creación.

- **Creación de bloques:** La clase *BlockFactory* permite la instanciación de un *BlockModel* (sea un bloque real en el *Workspace* o una plantilla en la *Toolbox*). Durante este proceso, se crean las *ConnectionModel* (*OutputConnection*, *PreviousConnection*, *NextConnection*) y las *InputModel* (que a su vez contienen *FieldModel* y sus propias *ConnectionModel* de entrada), asignando siempre la referencia correcta al bloque propietario (*SourceBlock*).
- **Eliminación de bloques:** La eliminación se coordina a través del método *BlockModel.Dispose()*. Este método no solo retira el bloque de las bases de datos del *Workspace*, sino que también se encarga de desvincularlo de sus conexiones mediante *BlockModel.UnPlug()* y de eliminar recursivamente todos sus bloques hijos conectados (liberando así los recursos asociados a la vista y al modelo del bloque). Este proceso garantiza una limpieza efectiva de la memoria y la coherencia del estado del programa.

3.3. Estructura Lógica de los Bloques (*BlockModel*)

Cada bloque es una unidad fundamental del programa, representada por una instancia de *BlockModel*. Un *BlockModel* encapsula la información lógica y las interacciones de un bloque.

- **Componentes principales de un *BlockModel*:**
 - **Connection:** Los puntos de conexión lógicos del bloque.
 - **OutputConnection:** Permite que un bloque que devuelve un valor (ej. número o booleano) se conecte a la entrada de otro.
 - **PreviousConnection:** Enlaza el bloque a una pila superior de sentencias (bloque anterior).
 - **NextConnection:** Conecta este bloque al siguiente en una secuencia lineal de sentencias (bloque inferior en la pila).

- **InputList:** `List<InputModel>` que define las entradas visuales del bloque, que a su vez contienen campos o conexiones para otros bloques.
- **ChildBlocks:** `List<BlockModel>` de los bloques lógicamente anidados bajo este bloque (e.g., bloques conectados a entradas de valor o a entradas de sentencia en un bucle).
- **Data:** Datos adicionales asociados al bloque (ej. configuración interna de un tipo de bloque).
- **Propiedades de estado:** (`Disabled`, `Movable`, `Editable`, `IsShadow`, `Collapsed`, `InputsInline`) controlan el comportamiento y apariencia del bloque.

Jerarquía Interna y Composición de un Bloque

La estructura de un programa se construye a partir de la jerarquía de `BlockModels`. Un bloque puede ser parte de una cadena lineal de sentencias o actuar como contenedor para otros bloques hijos.

- Un **Bloque** (`BlockModel`) se sitúa en la cima de su árbol jerárquico dentro del `Workspace` (un `TopBlock`), o es parte de un árbol más grande.
- Las **Conexiones Principales** (`OutputConnection`, `PreviousConnection`, `NextConnection`) vinculan el bloque a otros, definiendo su lugar en la pila o el flujo de valores.
- Los `InputModel` (Entradas) de un bloque actúan como puntos de acoplamiento para otros bloques (`Input Connections`) o como contenedores para **Fields** (Campos), organizando la disposición visual de los elementos internos.
- Un `BlockModel` puede tener otros `BlockModels` conectados jerárquicamente a través de sus entradas, constituyendo sus **ChildBlocks**.

3.4. Conexiones (`ConnectionModel`)

Las conexiones son el mecanismo central mediante el cual los bloques se unen entre sí, formando la estructura lógica del programa. En UBlockly, cada punto de unión se representa por una instancia de `ConnectionModel`.

Implementación de `ConnectionModel`

La clase `ConnectionModel` verifica la compatibilidad entre conexiones y gestiona sus estados y relaciones. Es fundamental para el proceso de arrastre y snap (conexiones) de bloques en la interfaz.

■ Funcionalidad principal:

- **Verificación de compatibilidad:** Determina si dos conexiones son lógicamente compatibles según su tipo y las propiedades de `Check`. (`CanConnectWithReason()`).
- **Conexión y desconexión:** Proporciona métodos para establecer y romper la relación entre bloques (`Connect()`, `Disconnect()`).
- **Manejo de eventos:** Notifica a sus observadores (`ConnectionView`) sobre cambios de estado (`UpdateState.Connected`, `Disconnected`, `Highlight`, `BumpedAway`).
- **Lógica de Bumping:** Implementa la capacidad de mover bloques ya conectados cuando un nuevo bloque intenta unirse a una conexión ocupada, reorganizando las pilas existentes (`ConnectionModel.Connect()` y métodos auxiliares).

■ Atributos principales:

- **SourceBlock:** `BlockModel` al que pertenece esta conexión. Crucial para la gestión de la jerarquía y la pertenencia a un `Workspace`.
- **Type:** Tipo de la conexión (`EConnection`): `InputValue`, `OutputValue`, `NextStatement`, `PrevStatement`, o `DummyInput` (cuando el `Input` no lleva conexión).
- **Location:** Posición *lógica* de la conexión en coordenadas del espacio de trabajo (`Vector2`), que se sincroniza con su posición visual para permitir la detección de snap entre bloques.
- **TargetConnection:** Referencia a la `ConnectionModel` a la que este bloque está unido. Es `null` si no está conectado.
- **IsConnected:** `bool` que indica si la conexión está activa.
- **Check:** `List<string>` que especifica los tipos de valores compatibles con esta conexión (e.g., `Number`, `Boolean`, `Statement`).
- **InDB:** `bool` que indica si la conexión está actualmente registrada en las bases de datos de conexión del `Workspace` (`ConnectionDBList`) para búsquedas.

- **DB / DBOpposite:** Referencias directas a las `BlockConnectionDB` correspondientes al tipo de esta conexión (DB) y al tipo opuesto (DBOpposite), para búsquedas rápidas.

3.5. Entradas (`InputModel`)

Un `InputModel` representa una entrada dentro de un bloque, ya sea para anidar otros bloques o para contener campos. Gestiona la alineación y organización de los diferentes elementos visuales dentro del bloque y su contribución al layout.

■ Atributos principales:

- **Name:** Nombre único de la entrada dentro de su bloque propietario (ej. STEPS, DO).
- **Type:** Tipo de la entrada (`EConnection`), indicando si es de valor (`InputValue`), de sentencia (`NextStatement`) o dummy (`DummyInput`).
- **Connection:** Si la entrada es de tipo `InputValue` o `NextStatement`, esta referencia a `ConnectionModel` permite anidar otro bloque dentro del actual.
- **FieldRow:** `List<FieldModel>` que almacena los campos (*Fields*) o etiquetas de texto asociados a esta entrada, definiendo su apariencia interna.
- **SourceBlock:** `BlockModel` padre al que pertenece esta entrada, garantizando una referencia coherente en la jerarquía del modelo.
- **ConnectedBlock:** Si `Connection` está activa, esta propiedad retorna el `BlockModel` que está conectado a esta entrada.
- **Align:** Determina la alineación horizontal de los campos dentro de la entrada (`EAlign.Left`, `Center`, `Right`).

3.6. Campos (`FieldModel`)

Los `FieldModels` representan los datos y etiquetas individuales dentro de un bloque. Son elementos visuales editables o estáticos que muestran información relevante para la configuración del bloque.

■ Atributos principales:

- **Name:** Identificador único del campo dentro de su bloque.
- **Type:** Tipo del campo (`field_label`, `field_input`, `field_number`, `field_dropdown`, etc.), utilizado por la `FieldFactory` para su instanciación.
- **mText:** Valor textual actual del campo.
- **SourceBlock:** `BlockModel` al que se encuentra asociado este campo.
- **IsEditable:** `bool` que indica si el campo permite al usuario modificar su valor en la interfaz.
- **IsImage:** `bool` que indica si el campo contiene una imagen en lugar de texto (ej. `field_image`).
- **Validator:** Función delegada (`FieldValidator`) que se utiliza para aplicar reglas de validación al valor de entrada del usuario, garantizando la consistencia de los datos.

3.7. Variables (*VariableMap*)

La clase `VariableMap` es el componente del `WorkspaceModel` que gestiona la creación, eliminación, renombrado y almacenamiento de las variables que los usuarios definen en el entorno de programación.

▪ **Atributos clave:**

- **mVariableMap:** `Dictionary<string, List<VariableModel>` que agrupa las variables por su tipo (cadena, número, etc.), permitiendo un acceso y una gestión eficientes.
- **mWorkspace:** Hace referencia a la instancia del `WorkspaceModel` asociada, garantizando la pertenencia y el contexto operativo del mapa de variables.

3.8. Sistema de Ejecución (*CSharpRunner* e *Cmdtors*)

El corazón de la lógica programable reside en el sistema de ejecución, responsable de interpretar y ejecutar la secuencia de bloques del usuario. En el prototipo, esto se realiza mediante un intérprete directo basado en corrutinas de C#, orquestado por `CSharpRunner` y los intérpretes de bloques (`Cmdtors`).

Clases del sistema de ejecución

- **Cmdtor:** Clase base abstracta para todos los intérpretes de bloques. Cada **Cmdtor** (o sus subclases: **ValueCmdtor**, **VoidCmdtor**, **EnumeratorCmdtor**) define cómo un tipo de bloque específico traduce su lógica a acciones programáticas. En el prototipo, **MotionBlockInterpreter** es un ejemplo concreto de esta implementación para el bloque de movimiento.
- **CmdEnumerator:** Clase auxiliar que envuelve un **IEnumerator** de un **Cmdtor**, permitiendo que el **CSharpRunner** lo gestione como una corrutina en una pila. Facilita la ejecución secuencial y la pausa/reanudar de las operaciones de los bloques.
- **CSharpRunner:** Componente **MonoBehaviour** que orquesta la ejecución del programa. Mantiene una pila de corrutinas (**Stack<IEnumerator>**) para manejar el flujo de ejecución (secuencial, anidado, condicionales, bucles) de los bloques. Permite el control del ciclo de vida del programa (inicio, pausa, reanudación, parada y modo paso a paso). También se encarga de delegar el inicio y fin de las corrutinas a la capa de visualización (**BlockObserver**) para la animación del robot y la interfaz de usuario.

3.9. Representación Visual y Composición de Bloques en Unity

Para adaptar UBlockly a Unity, cada **BlockModel** (la representación lógica de un bloque) se traduce en un **GameObject** con componentes **UnityEngine.UI** que conforman su *vista*. La **BlockView** (**BaseView**) es el componente principal que gestiona esta representación visual y sincroniza el modelo lógico con la interfaz de usuario. Este proceso de adaptación busca replicar la apariencia y el comportamiento de los bloques de Scratch, combinando la flexibilidad de UBlockly con una estética intuitiva.

Creación de Prefabs para Bloques en Unity

En Unity, un **Prefab** es un tipo de activo que permite almacenar un **GameObject** con sus componentes, propiedades y toda su jerarquía de hijos como una plantilla reutilizable. Esta aproximación facilita la creación consistente y el mantenimiento de múltiples instancias de un mismo objeto en una escena o a través del tiempo. UBlockly ofrece métodos tanto para generar los

bloques de manera procedimental en tiempo de ejecución como para utilizar prefabs prediseñados. Para el desarrollo de este prototipo, se ha optado por un enfoque que combina el uso de prefabs configurados manualmente en el editor de Unity con la flexibilidad de la estructura lógica de UBlockly. Esta decisión permite un control granular sobre la apariencia y el comportamiento visual exacto de cada bloque, asegurando que el prototipo se ajuste a la experiencia visual deseada de Scratch. Cada tipo de bloque (como, por ejemplo, mover X pasos) se ha diseñado como un prefab distinto, preconfigurado con su estructura visual base y componentes de `UnityEngine.UI`.

Ejemplo de Composición de Bloque: `motion_movesteps`

Un claro ejemplo de la composición visual de los bloques mediante prefabs es el bloque `motion_movesteps`, ilustrado en la Figura 3.1. Este prefab representa el bloque mover 10 pasos y muestra su estructura interna en la jerarquía de Unity, que refleja el anidamiento de componentes `BlockView`, `LineGroupView`, `InputView` y `FieldView` que construyen el bloque:

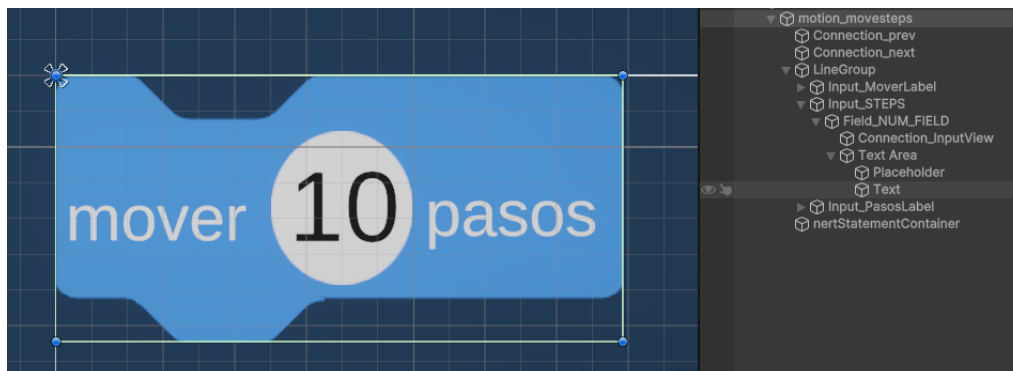


Figura 3.1: Estructura del Prefab `motion_movesteps` en la Jerarquía de Unity.

La jerarquía interna de este prefab es clave para su funcionamiento y composición:

- **`motion_movesteps`:** Es el `GameObject` raíz del prefab, que contiene el script `BlockView`. Este `BlockView` es responsable de sincronizar el modelo lógico (`BlockModel`) con la interfaz visual.
- **`Connection_prev` y `Connection_next`:** Hijos directos que representan visualmente los puntos de conexión anterior y siguiente, gestionados por componentes `ConnectionView`.

- **LineGroup:** Un `GameObject` hijo que agrupa los elementos internos de la línea principal del bloque (campos de texto, entradas). Contiene un componente `LineGroupView` que organiza el layout horizontal de estos elementos.
- **Input_MoverLabel:** Un `GameObject` hijo dentro de `LineGroup` que representa la etiqueta de texto mover. Contiene un componente `InputView` (`Input_MoverLabel`) que gestiona el texto y sus campos. A su vez, dentro de él, se encuentran las `FieldView` que contienen la información de texto (mover).
- **Input_STEPS:** Otro `GameObject` hijo dentro de `LineGroup` que representa la entrada numérica editable (por defecto 10). Contiene un `InputView` (`Input_STEPS`) y dentro de este se anidan:
 - **Field_NUM_FIELD:** Componente `FieldView` que gestiona el valor numérico (inicialmente 10). Es un campo editable y en su jerarquía contiene los componentes visuales para mostrar y editar ese valor (`Text Area`, `Placeholder`, `Text`).
 - **Connection_InputView:** Contiene un componente `ConnectionView` que representa visualmente el punto de conexión circular para enlazar otros bloques de valor (ej. un número, una variable).
- **Input_PasosLabel:** Un `GameObject` dentro de `LineGroup` para la etiqueta de texto pasos. Funciona de manera análoga a `Input_MoverLabel`.
- **nextStatementContainer:** Este `RectTransform` es un hijo clave que no contiene un `InputView`, sino que actúa como **el punto de anclaje y contenedor visual para los bloques que se conectan por debajo** del bloque actual (`BlockModel.NextConnection`). Cuando un bloque (`BlockView`) se conecta a este `NextConnection`, su `GameObject` raíz es re-parentado a este `nextStatementContainer`. Esto es fundamental para que las pilas de bloques se apilen y se gestionen correctamente dentro de la interfaz gráfica, manteniendo la coherencia visual del programa.

Maapeo Modelo-Vista y Elementos UI

La sincronización entre la lógica de programación (el Modelo, `BlockModel`) y su presentación visual (la Vista, `BlockView`) es fundamental. Esta relación bidireccional se mantiene mediante un patrón Observador, donde los cambios en el modelo (`BlockModel.FireUpdate()`) notifican a la vista

(`BlockView.HandleModelUpdate()`) para que se actualice. A su vez, las interacciones del usuario en la vista son traducidas por los controladores para modificar el modelo.

- **Elementos UI comunes:** Los bloques se componen visualmente de una combinación de los siguientes componentes Unity UI, cada uno asociado a una clase **View** específica:
 - **Image:** Utilizado para la forma base del bloque, su color y sus muescas/pestañas características. Se utiliza un componente especializado (`CustomMeshImage`) que permite la manipulación de la geometría del bloque para adaptarla a la forma de las muescas, pestañas y conexiones de Scratch.
 - **TextMeshProUGUI:** Representan las etiquetas de texto de los bloques (ej. mover, pasos) y los valores numéricos o de texto editables por el usuario.
 - **Button:** Utilizado para elementos interactivos como campos de selección desplegables (`TMP_Dropdown` para `FieldDropDownView` y `FieldVariableView`) y posibles botones en el futuro (`FieldButtonView`).

Sistema de Posicionamiento y Tamaño (Layout Manual)

El correcto despliegue visual de los bloques en la interfaz requiere un sistema de layout preciso, dado que Unity UI no siempre ofrece un ajuste automático óptimo para las formas complejas y anidadas de los bloques de programación visual.

- **RectTransform, Anchor Points y Pivot:** Todos los elementos UI (`GameObjects`) de los bloques poseen un componente `RectTransform`. Su posicionamiento y redimensionamiento se gestionan a través de la manipulación programática de sus propiedades `anchoredPosition`, `pivot` y `anchor` en un **sistema de layout manual**. Este sistema calcula dinámicamente el tamaño (`CalculateSize()`) y la posición (XY) de cada `BlockView` y sus hijos (`InputView`, `FieldView`, `ConnectionView`) en la jerarquía visual de la UI.
- **Conexiones Visuales (ConnectionView):** Cada `ConnectionModel` tiene una `ConnectionView` asociada que representa visualmente el punto de conexión. Estas vistas gestionan el *resaltado visual* ('Highlight') para indicar posibles conexiones durante el arrastre, y son clave

en el *re-parentado y posicionamiento de bloques* tras un "snap". Aunque no se emplean `LineRenderer` o `Bezier Curves` para el trazado de las conexiones (como ocurre en otras herramientas de modelado de software), el snap visual y la animación son fundamentales para la experiencia de usuario.

3.10. Serialización y Persistencia

Para permitir guardar y cargar el estado de los programas del usuario, el prototipo incorpora un sistema de serialización y persistencia. Este sistema se encarga de almacenar la estructura lógica del `WorkspaceModel` y de restaurarla cuando sea necesario, permitiendo la continuidad en los proyectos.

- **Formato de datos:** La estructura de los bloques y sus relaciones se almacenan en formato **XML**, el cual permite una representación jerárquica clara del programa.
- **Clases involucradas:**
 - **Xml:** Componente del core de UBlockly responsable de transformar la estructura del `WorkspaceModel` a un árbol `XMLDocument` (`WorkspaceToDom()`) y viceversa (`DomToWorkspace()`).
 - **WorkspaceSerializer:** La clase implementada en el prototipo que utiliza las funcionalidades de `Xml` para *guardar* el XML en las `PlayerPrefs` de Unity y *cargar* programas desde ellas, proporcionando una persistencia básica para el estado del laboratorio.

3.11. Objetivos de Desarrollo Sostenible (ODS) y su Aplicación en el Proyecto

Los Objetivos de Desarrollo Sostenible (ODS) son una lista de 17 objetivos globales adoptados por la Asamblea General de la Organización de las Naciones Unidas (ONU) en 2015, como parte de la Agenda 2030 para el Desarrollo Sostenible. Estos objetivos universales e interconectados buscan abordar desafíos globales urgentes, como la pobreza, la desigualdad, el cambio climático, la degradación ambiental, la paz y la justicia. Su integración en proyectos de ingeniería informática refleja un compromiso con la creación de soluciones con un impacto positivo a nivel social, económico y ambiental.

El presente TFG se alinea con los ODS al fomentar una educación inclusiva y al reducir las desigualdades, mediante la democratización del acceso a herramientas de aprendizaje tecnológico.

ODS 4: Educación de Calidad

El **ODS 4: Educación de Calidad** busca Garantizar una educación inclusiva y equitativa de calidad y promover oportunidades de aprendizaje permanente para todos[20]. En el contexto de este proyecto, se contribuye a este ODS de las siguientes maneras:

- **Acceso equitativo:** El laboratorio virtual reduce significativamente las barreras de entrada a la educación en programación y robótica. Al eliminar la necesidad de hardware físico costoso y laboratorios especializados, el proyecto democratiza el acceso, permitiendo a estudiantes de zonas con recursos limitados o sin acceso a infraestructuras dedicadas, participar en experiencias de aprendizaje enriquecedoras.
- **Educación inclusiva:** Al ofrecer una interfaz basada en la programación por bloques tipo Scratch, accesible e intuitiva, se facilita el aprendizaje a niños y adolescentes con diversos niveles de experiencia y habilidades. El diseño del entorno está concebido para ser comprensible para un público amplio, desde principiantes hasta usuarios más avanzados.
- **Aprendizaje permanente:** La adquisición de habilidades en pensamiento computacional y lógico desde edades tempranas sienta una base sólida para el aprendizaje continuo en el ámbito de las tecnologías.

De esta forma, el proyecto es una herramienta que promueve activamente la participación de una comunidad estudiantil más amplia en las disciplinas STEM [20].

ODS 10: Reducción de las Desigualdades

El **ODS 10: Reducción de las Desigualdades** tiene como meta Reducir la desigualdad en los países y entre ellos [20]. La contribución del proyecto se manifiesta en los siguientes aspectos:

- **Democratización del acceso a la tecnología educativa:** El coste de la tecnología educativa es un factor determinante que a menudo

excluye a segmentos de la población. Al priorizar el desarrollo de una aplicación para dispositivos Android (como tablets de bajo coste) y utilizar software de código abierto (Unity Personal, C#, UBlockly), el proyecto minimiza la inversión económica requerida para acceder a estas herramientas didácticas.

- **Brecha digital y geográfica:** En regiones donde la infraestructura tecnológica y los recursos educativos son escasos, las simulaciones virtuales pueden compensar la falta de laboratorios físicos y equipamiento de robótica. Esto reduce la brecha digital y geográfica, permitiendo que estudiantes de entornos rurales o menos privilegiados accedan a la misma calidad de experiencia de aprendizaje que sus homólogos en áreas urbanas.

Este proyecto no solo facilita el aprendizaje técnico, sino que también sirve como una herramienta para fomentar una sociedad más justa e igualitaria al hacer la educación en programación y robótica más accesible a todos.

4. Técnicas y herramientas

El presente capítulo tiene como objetivo describir las técnicas metodológicas y las herramientas de desarrollo que se han utilizado para llevar a cabo el proyecto. Se detallarán los aspectos más destacados de cada elección, justificando el porqué de su utilización frente a posibles alternativas, y cómo estas herramientas han contribuido al logro de los objetivos planteados. Para mejorar la comprensión, las herramientas se han agrupado en subsecciones según su área de aplicación.

4.1. Metodología de Desarrollo: Agile con SCRUM

El desarrollo del prototipo de laboratorio virtual se ha abordado mediante la aplicación de principios de metodologías ágiles, en particular el marco de trabajo **SCRUM**. Este enfoque, a diferencia de modelos lineales o en cascada, ha sido fundamental por su énfasis en la flexibilidad, la colaboración y la entrega continua de valor. Permite una rápida adaptación a los desafíos técnicos y a la evolución de los requisitos durante el desarrollo de un proyecto complejo y en evolución como este.

- **Planificación Iterativa (Sprints):** El desarrollo se ha estructurado en *sprints* de duración fija (ej. 1-3 semanas), al final de los cuales se obtenía un incremento funcional y probado del prototipo.
- **Roles y Colaboración:** El rol de tutor (D. Carlos López Nozal) ha adoptado las funciones de *Product Owner* (priorizando funcionalidades y validando incrementos) y *Scrum Master* (guiando la aplicación de la

metodología). El rol del alumno (autor del TFG) ha sido el de *Equipo de Desarrollo*.

- **Monitorización del Progreso:** La plataforma **Zube** [24] ha sido utilizada para la gestión de proyectos, integrándose con GitHub y permitiendo el uso de tableros Kanban para planificar tareas, asignar puntos de historia y realizar seguimiento visual del progreso.
- **Integración Continua y Calidad del Código:** Como parte del compromiso con la calidad del software, se han explorado prácticas de Integración Continua (CI), aunque no completamente automatizadas, con revisiones periódicas del código en GitHub y una inspección básica de las estructuras y coherencia implementadas, detectando posibles errores y problemas de estilo o rendimiento.

Esta metodología ha permitido validar el prototipo de forma continua con la funcionalidad principal (movimiento del robot) e identificar los desafíos para las fases futuras de forma temprana, asegurando que el esfuerzo se centrara en las áreas de mayor impacto para el TFG.

4.2. Herramientas Core para el Desarrollo de Software

Motor de Videojuegos: Unity 6 y Lenguaje C#

El corazón del laboratorio virtual reside en el motor de videojuegos **Unity Versión: 6000.0.26f1**, elegido por su potente capacidad para la creación de entornos 3D interactivos y simulaciones en tiempo real. La decisión de utilizar Unity, frente a motores exclusivamente de renderizado, se fundamenta en sus capacidades para:

- **Simulación 3D avanzada:** Permite el modelado, animación y simulación de la física del robot, esenciales para replicar comportamientos realistas.
- **Sistema de UI unificado (Unity UI Canvas):** Proporciona un marco robusto para construir interfaces de usuario complejas mediante un **Unity UI Canvas (UICanvasView)**, lo que ha facilitado la creación del editor de bloques con su interacción visual.

- **Compatibilidad multiplataforma:** Soporta la exportación a diversos sistemas operativos, incluyendo **Android (Tablets)**, que es la plataforma objetivo del prototipo, lo cual es crítico para la accesibilidad del proyecto.
- **Comunidad y recursos extensivos:** Dispone de una vasta comunidad de desarrolladores y una amplia biblioteca de recursos que agilizan el desarrollo.

El lenguaje de programación principal utilizado es **C#**. La integración nativa de **C#** con Unity permite el desarrollo de scripts eficientes basados en programación orientada a objetos (POO), y el uso intensivo de **corrutinas** (**IEnumerator**) para la gestión de la lógica asíncrona de la ejecución de bloques y animaciones, crucial para una experiencia fluida del usuario. El entorno de desarrollo integrado (IDE) principal ha sido **Visual Studio**.

Modelado 3D y Creación de Modelos de Robot

Para la creación del modelo 3D del robot y la representación de componentes específicos de LEGO WeDo 2.0, se han utilizado herramientas de modelado dedicadas:

- **Blender (Versión 4.4):** Como software de modelado 3D principal. Blender permitió diseñar un modelo de robot de forma personalizada, optimizar la malla para asegurar un rendimiento eficiente en tiempo real en Unity, y exportar los modelos en formatos compatibles.
- **LEGO Digital Designer (LDD) o Studio 2.0:** Como herramientas auxiliares, han sido útiles para entender la composición de los robots LEGO WeDo 2.0 y para generar ideas de construcción de modelos 3D que repliquen su estética, sirviendo como referencia para el modelado en Blender.

Esta fase ha sido fundamental para transformar la idea de un robot LEGO WeDo 2.0 virtual en un activo 3D funcional dentro de Unity, asegurando la cohesión visual del proyecto.

Framework de Interfaz de Programación: UBlockly

El núcleo de la interfaz de programación visual del prototipo se basa en la adaptación de la librería **UBlockly** [9], una implementación de Blockly

en C# para Unity. Esta elección, en lugar de un desarrollo desde cero, se justifica por:

- **Base Lógica Sólida:** UBlockly proporciona una estructura ya desarrollada para la representación de bloques como modelos lógicos (`BlockModel`, `InputModel`, `FieldModel`) y la gestión de sus conexiones (`ConnectionModel`, `BlockConnectionDB`), lo que acelera el desarrollo.
- **Motor de Ejecución Propio:** Incluye un sistema de intérpretes directos basados en clases `Cmdtor` y la orquestación de corrutinas con `CSharpRunner`, fundamental para traducir la lógica de bloques en acciones ejecutables en Unity.
- **Extensibilidad y Personalización:** La arquitectura de UBlockly es altamente modular, permitiendo la adaptación a las necesidades de este proyecto. Se ha extendido para replicar las funcionalidades y la estética de Scratch, adaptando sus componentes a un entorno MVC y facilitando la creación de bloques específicos para el robot (ej. `motion_movesteps`).
- **Adaptación visual a Unity UI:** Se han creado **prefabs** de `GameObjects` que contienen las `BlockViews` personalizadas con componentes `UnityEngine.UI` (`Image`, `TextMeshProUGUI`, `Button`), que se corresponden directamente con los `BlockModels` lógicos. Esto ha permitido controlar al detalle la apariencia y comportamiento de los bloques en la interfaz. Elementos clave como el `nextStatementContainer` dentro de la jerarquía de los prefabs, son puntos de anclaje visuales para asegurar la correcta apilación de los bloques.

4.3. Persistencia de Datos del Programa

Para permitir guardar y cargar los programas creados por el usuario, el prototipo incorpora un sistema de persistencia básico.

Serialización a XML y Almacenamiento en PlayerPrefs

El estado del programa, representado por la estructura lógica del `WorkspaceModel` (que incluye todos los `BlockModels`, sus conexiones y sus datos), se serializa a formato **XML**. La clase `Xml` de UBlockly se encarga de esta transformación del modelo a XML y viceversa (`WorkspaceToDom()` y `DomToWorkspace()`). La clase `WorkspaceSerializer` de la solución desarrollada utiliza estas

funcionalidades para *guardar el XML resultante en las `PlayerPrefs`* de Unity y *cargarlo* desde ahí. Esta solución es adecuada para un prototipo, dada su simplicidad y su funcionalidad de almacenamiento local rápido y básico para la continuidad en los proyectos de usuario.

4.4. Herramientas para Gestión de Proyectos y Desarrollo

Control de Versiones: Git y GitHub

El control de versiones es fundamental para el desarrollo colaborativo y el seguimiento de los cambios en el código. **Git** ha sido utilizado como sistema de control de versiones distribuido, permitiendo un registro detallado del historial del proyecto y la gestión eficiente de las distintas ramas de desarrollo. El repositorio en **GitHub** ha servido como plataforma de alojamiento remoto, facilitando la colaboración, el versionado y la sincronización del código fuente. Sus características, como la gestión de **issues** y **milestones** han sido valiosas para estructurar y monitorizar el avance del proyecto.

Asistencia en Codificación: GitHub Copilot

Durante la fase de implementación, se ha hecho uso de **GitHub Copilot**, una herramienta de asistencia en la codificación basada en Inteligencia Artificial [8]. Copilot, que se integra con IDEs como Visual Studio, ofrece sugerencias de código en tiempo real, completado automático y la generación de fragmentos de código basados en el contexto. Esto ha contribuido a aumentar la eficiencia del desarrollo, reduciendo la necesidad de escribir código repetitivo y facilitando la exploración de patrones de programación. Su aplicación se ha limitado a tareas de apoyo, manteniendo el control total del código en manos del desarrollador.

Edición y Compilación de Documentación: TeXstudio y LaTeX

La documentación de la memoria del Trabajo Fin de Grado ha sido elaborada utilizando **LaTeX** como sistema de composición de textos, conocido por su capacidad para generar documentos de alta calidad tipográfica y su robustez en la gestión de referencias bibliográficas. La herramienta **TeXstudio** ha actuado como Entorno de Desarrollo Integrado (IDE) para LaTeX, facilitando la escritura, la corrección ortográfica y gramatical, el

resaltado de sintaxis y la previsualización en tiempo real del documento. Esta combinación ha sido esencial para la producción de una memoria técnica profesional. .

5. Aspectos relevantes del desarrollo del proyecto

Este capítulo tiene como objetivo recoger los aspectos más interesantes y no triviales del proceso de desarrollo del prototipo del laboratorio virtual de programación y robótica. Se detallarán las decisiones de diseño arquitectónico y de implementación clave, justificando los caminos adoptados y las estrategias empleadas para resolver los desafíos técnicos, especialmente aquellos inherentes a la integración de UBlockly en Unity y la simulación del comportamiento de los bloques. Se busca que este apartado sirva como un resumen de la experiencia práctica del proyecto y una referencia útil para futuros desarrolladores.

5.1. Metodología de Desarrollo: Iteraciones Ágiles con SCRUM

El proceso de desarrollo de este prototipo ha seguido una metodología adaptativa, inspirada en los principios de **SCRUM**. Esta elección, frente a modelos más lineales, se justificó por la naturaleza experimental del proyecto y la necesidad de una *respuesta rápida* a los desafíos tecnológicos emergentes. La aplicación de este marco ágil permitió:

- **Entrega de Valor Incremental:** El proyecto se estructuró en ciclos de *sprints* de duración fija (ej., una a tres semanas), donde al final de cada iteración se obtenía un incremento funcional y probado del prototipo. Esto aseguró un progreso tangible y la validación temprana

de funcionalidades críticas, como el movimiento del robot, que era la piedra angular del prototipo.

- **Flexibilidad y Adaptación Continua:** La revisión constante del progreso con el tutor (asumiendo roles de *Product Owner* y *Scrum Master*) permitió reevaluar prioridades, adaptar requisitos y refinar las soluciones técnicas en función de la experiencia práctica obtenida, así como gestionar la complejidad que surgió de la interacción entre UBlockly y Unity.
- **Transparencia y Gestión del Trabajo:** Herramientas como **Zube** [24] se emplearon para la planificación de *sprints*, el seguimiento de tareas con tableros *Kanban* y la asignación de *puntos de historia*. Esto facilitó la visualización del trabajo pendiente y realizado, mejorando la organización del proyecto.

Este enfoque iterativo fue clave para la gestión eficaz de la complejidad técnica del proyecto y para alcanzar un *Producto Mínimo Viable (MVP)* dentro del plazo establecido.

5.2. Decisiones de Diseño Arquitectónico e Implementación

Adopción de la Arquitectura Modelo-Vista-Controlador (MVC)

Una de las decisiones arquitectónicas fundamentales fue la adopción de un patrón **Modelo-Vista-Controlador (MVC)**. Esta elección es crítica en sistemas complejos como los entornos de desarrollo integrados, donde una separación clara de responsabilidades es esencial para la mantenibilidad, la escalabilidad y la depuración. El MVC se ha implementado de la siguiente manera:

- **Modelo (Lógica de Negocio y Datos Puros):** Las clases centrales del Modelo (`WorkspaceModel`, `BlockModel`, `ConnectionModel`, `InputModel`, `FieldModel`, `VariableMap`, `ProcedureDB`) encapsulan el estado lógico y las reglas de negocio del sistema de bloques. Son independientes de cualquier representación visual y se enfocan en la *abstracción de la programación visual* del usuario.

- **Vista (Representación Visual e Interacción UI):** Componentes Unity UI como `UnityEngine.UI.Image`, `TMPPro.TextMeshProUGUI`, `UnityEngine.UI.Button` actúan como los elementos visuales, gestionados por una jerarquía de clases `View` (`BaseView`, `BlockView`, `ConnectionView`, `InputView`, `FieldView`, `LineGroupView`). Estas vistas son las únicas que conocen la interfaz de Unity y se encargan de representar el Modelo.
- **Controlador (Orquestación e Interpretación de Interacciones):** Capas de lógica como `AppController`, `WorkspaceController`, `BlockDragController`, `BlockConnectionController`, `ExecutionController` e `InputController` median entre el Modelo y la Vista. Interpretan las acciones del usuario, validan operaciones y orquestan las actualizaciones del Modelo y la Vista sin que estas capas se conozcan directamente entre sí.

La comunicación entre estas capas se gestiona primordialmente mediante el **patrón Observador** (`Observable<TArgs>` e `IObserver<TArgs>`). Los modelos (`BlockModel`, `ConnectionModel`, `FieldModel`) actúan como sujetos observados, notificando de cualquier cambio relevante a sus vistas o controladores suscritos. Esto asegura una *sincronización eficiente* de la interfaz sin acoplamientos rígidos entre los elementos. Por ejemplo, cuando un usuario cambia el valor de un campo (`FieldView`), este notifica al controlador (`InputController`), que solicita al modelo (`FieldModel`) el cambio, el cual a su vez notifica a la vista (`FieldView`) para que se actualice.

Aprovechamiento y Adaptación del Repositorio UBlockly

La decisión de basar la implementación del motor de bloques en el repositorio de **UBlockly** [9], una adaptación de Blockly en C# para Unity, fue una *elección estratégica* frente al desarrollo desde cero. Este enfoque se justificó por la necesidad de construir un prototipo funcional robusto en un tiempo limitado, ya que UBlockly proporcionó una base sólida para:

- **Motor Lógico de Bloques:** Sus clases core como `WorkSpaceModel` (gestor principal de la base de datos de bloques), `BlockModel` (estructura lógica de un bloque) y `ConnectionModel` (sistema de conexiones entre bloques) ofrecieron una base abstracta y funcional ya testada. Se adoptó este diseño tal cual, centrándose la adaptación en cómo estos modelos interactúan con la nueva capa de vistas en Unity.

- **Motor de Ejecución Asíncrono:** UBlockly ya incluye un robusto sistema de interpretación directa basado en `Cmdtors` y corrutinas de C# (`CSharpRunner`). Esta arquitectura se ha adoptado directamente, adaptando los intérpretes (`MotionBlockInterpreter`) para interactuar con los objetos simulados del robot.
- **Serialización y Persistencia:** El mecanismo de UBlockly para serializar el estado del `Workspace` a XML y viceversa se integró directamente, proporcionando una solución eficiente para guardar y cargar los proyectos del usuario.

La adaptación principal ha consistido en *acoplar las capas de modelo y ejecución de UBlockly con la nueva capa de visualización de Unity UI*, así como *customizar la lógica de interacción* (drag & drop) para lograr una experiencia más cercana a Scratch. Se han creado prefabs de `GameObjects` de Unity con la estructura de componentes `View` que reflejan los `BlockModels` lógicos, permitiendo la creación modular y una apariencia flexible de los bloques.

5.3. Detalles Clave de Implementación

Representación Visual y Composición de Bloques en Unity

Uno de los mayores desafíos en el desarrollo de interfaces de programación visual es la correcta representación y sincronización entre el modelo lógico del programa y su visualización interactiva. En este proyecto, la aproximación se basa en la creación de **prefabs** de `GameObjects` de Unity para cada tipo de bloque (`BlockView`), replicando la estética de Scratch.

- **Creación de Prefabs Estructurados:** Cada `BlockView` es un prefab con una jerarquía de componentes `UnityEngine.UI` que se corresponden directamente con las estructuras lógicas del `BlockModel` (ej., `InputView` para entradas, `FieldView` para campos). Por ejemplo, el bloque `motion_movesteps` es un prefab con un `GameObject` `motion_movesteps` (con `BlockView` y `Image`) que agrupa lógicamente los hijos (`LineGroup`, `Connection_prev`, `Connection_next`, `nextStatementContainer`). Esta composición predefinida de prefabs acelera la instanciación de bloques en tiempo de ejecución.

- **Layout Manual y Sincronización de `RectTransform`:** A diferencia de los sistemas de layout automático de Unity, el proyecto emplea un **sistema de layout manual** (`BaseView.UpdateLayout()`) para los bloques. Este sistema calcula dinámicamente la posición (XY) y el tamaño (`CalculateSize()`) de cada `BlockView` y sus subcomponentes (`InputView`, `FieldView`, `ConnectionView`). Es crucial porque los bloques de programación visual tienen formas complejas (muescas, pestañas, huecos para entradas) que no se ajustan de manera óptima a layouts grid o verticales/horizontales simples. Cada vez que el modelo lógico de un bloque cambia de posición (`BlockModel.XY`), o su contenido o tamaño (`BlockModel.FireUpdate()` para `UpdateStates.Inputs` o `UpdateStates.Connections`), el `BlockView` es notificado y marca su layout como sucio (`'NotifyLayoutDirty'`). Posteriormente, en un ciclo de Unity, se fuerza un recálculo del layout completo o parcial mediante `LayoutRebuilder.ForceRebuildLayoutImmediate()`.
- **Visualización de Conexiones e Identificación de `nextStatementContainer`:** Las conexiones lógicas (`ConnectionModel`) tienen su reflejo visual en las `ConnectionViews`. Un elemento fundamental en el diseño de los prefabs es el `RectTransform` denominado `nextStatementContainer` (ver jerarquía en la imagen adjunta en Capítulo 3 para `motion_movesteps`). Este `GameObject` no solo es un punto de anclaje visual, sino el **contenedor principal para la pila de bloques subsiguientes** conectados al `NextConnection`. Cuando un bloque se une, su `BlockView` raíz se re-parenta a este `nextStatementContainer`, asegurando la correcta representación jerárquica y el apilamiento visual de las sentencias.

Interacción con los Bloques: El Sistema de Arrastre y Conexión (Drag & Drop)

La funcionalidad de arrastrar y conectar bloques de forma intuitiva y fluida es central para replicar la experiencia de Scratch. Este sistema, orquestado por `BlockDragController` y `BlockConnectionController`, representa una implementación compleja con diversas interacciones:

- **Inicio del Arrastre:** Se gestiona desde un `BlockTemplateDragSource` para las plantillas de la `Toolbox`, o directamente desde la `BlockView` para bloques ya en el área de codificación. Este proceso implica la clonación del `BlockModel` (si es una plantilla), la creación de su `BlockView` asociada y su re-parentado a una capa superior (`'DragLayer'`) para la

interacción flotante. Se guarda un offset (`'m_dragStartOffset'`) para mantener la posición del puntero relativa al bloque.

- **Búsqueda y Resaltado de Conexiones:** Durante el arrastre, el `BlockDragController` solicita a `BlockConnectionController.ProcessDrag()` la identificación de posibles conexiones válidas. Este proceso implica:
 - Recorrer todas las `ConnectionModels` del bloque que se está arrastrando.
 - Utilizar las `BlockConnectionDBs` (`ConnectionDBList` en el `WorkspaceModel`) para buscar eficientemente las conexiones compatibles más cercanas (`SearchForClosest()`) a los puntos de conexión del bloque arrastrado. Estas bases de datos en memoria están optimizadas espacialmente para la búsqueda de vecinos cercanos, manteniendo la `Location` (`Vector2`) actualizada para cada `ConnectionModel`.
 - Aplicar reglas de validación estricta (`IsConnectionAllowed()`) para determinar si una conexión es posible, incluyendo compatibilidad de tipos, prevención de bucles jerárquicos y distancia límite (snap).
 - Si se encuentra un candidato, la `ConnectionView` correspondiente es instruida para activar un *resaltado visual* (`'Highlight'`), indicando al usuario la posible unión.
- **Conexión Lógica y Re-parentado Visual (Snap):** El punto más crítico se da al soltar el bloque (`BlockDragController.HandleEndDrag()`). Si existe una conexión válida:
 - **Conexión Lógica:** Las dos `ConnectionModels` (del bloque arrastrado y del estático) se unen mediante `ConnectionModel.Connect()`. Este método es fundamental por su implementación de la *lógica de bumping*: si una conexión receptora ya está ocupada, el bloque previamente conectado se *desconecta y se desplaza* (`'UpdateState.BumpedAway'`) a una posición cercana, y se busca una nueva ubicación para él dentro de la jerarquía de bloques (`'ConnectionModel.LastConnectionInRow()'`).
 - **Re-parentado Visual:** La `BlockView` del bloque conectado se re-parenta inmediatamente al `RectTransform` adecuado de su nuevo padre visual (ej. `nextStatementContainer`), ajustando su posición `anchoredPosition` para que encaje visualmente.
 - **Actualización de Layouts:** Tras cualquier conexión o desconexión, el sistema marca los `RectTransforms` afectados para una

reconstrucción del layout, asegurando que la interfaz se adapte visualmente al nuevo estado del programa (`LayoutRebuilder.MarkLayoutForRebuild()`).

El Motor de Ejecución de Bloques: De la Lógica a la Acción del Robot

El corazón operativo del prototipo es el motor de ejecución de bloques, responsable de interpretar la secuencia de instrucciones del usuario y traducirlas en acciones en el entorno de simulación 3D.

- **Intérpretes de Bloques (Cmdtors):** Cada tipo de bloque funcional (ej. movimiento) tiene una clase intérprete concreta que hereda de `Cmdtor` (`ValueCmdtor`, `VoidCmdtor`, `EnumeratorCmdtor`). Por ejemplo, `MotionBlockInterpreter` se encarga de extraer los parámetros de mover N pasos de un `BlockModel`.
- **Orquestación con CSharpRunner:** La clase `CSharpRunner`, un `MonoBehaviour`, es el orquestador principal de la ejecución. Mantiene una `Stack<IEnumerator>` de `CmdEnumerators` para gestionar el flujo de control (secuencia lineal, bucles, condicionales). Esto permite una *ejecución paso a paso no bloqueante* del programa.
- **Concurrencia con Corrutinas:** La característica distintiva de este sistema es el uso intensivo de las **corrutinas de C#** (`IEnumerator`, `yield return`). Cuando un intérprete de bloque (`Cmdtor`) invoca una acción que toma tiempo (ej., una animación de movimiento del robot, un retardo), este se traduce en un `yield return` en C#. El `CSharpRunner` *pausa la ejecución de ese bloque* y la *reanuda automáticamente* cuando la corrutina de la acción externa ha finalizado. Esto es crucial para lograr *animaciones fluidas* del robot o *retrasos precisos* sin congelar la aplicación. Por ejemplo, `MotionBlockInterpreter` llama a `BlockObserver._MoveRobot()` (una corrutina), y el `CSharpRunner` espera a que esta corrutina termine antes de pasar al siguiente bloque.
- **Sincronización con la Simulación:** La clase `BlockObserver` actúa como un puente entre el motor de ejecución de bloques y el `GameObject` del robot simulado (`RobotBehaviour`). `BlockObserver` recibe las órdenes de alto nivel (ej. mover 10 pasos) desde los `Cmdtors` y delega estas acciones a `RobotBehaviour` (el script que controla el movimiento y la animación física del robot), asegurando que la simulación refleje con precisión la lógica del programa. `BlockObserver` también gestiona el

feedback visual en la interfaz de usuario, como los mensajes de estado y el resaltado del bloque en ejecución.

Persistencia de Proyectos del Usuario

La capacidad de guardar y cargar los programas creados por el usuario es esencial para la continuidad del aprendizaje y la gestión de proyectos. El prototipo implementa un sistema básico pero funcional de persistencia.

- **Serialización en Formato XML:** La estructura lógica completa del `WorkspaceModel` (incluyendo todos los bloques, sus jerarquías internas y las conexiones establecidas) se serializa a un formato **XML**. Las clases `Xml` de `UBlockly` (`WorkspaceToDom()` y `DomToWorkspace()`) son el pilar de este proceso, transformando el árbol de objetos `C#` del modelo a un árbol XML y viceversa. Esta elección del formato XML proporciona una representación clara y legible de los programas.
- **Almacenamiento Local con `PlayerPrefs`:** La clase `WorkspaceSerializer`, desarrollada para este proyecto, es la encargada de manejar la persistencia del XML generado. Opta por *guardar el XML resultante directamente en las `PlayerPrefs`* de Unity y *cargarlo* desde esta ubicación local. Esta solución, aunque simple, es altamente eficaz para un prototipo, dada su facilidad de implementación y su funcionalidad de almacenamiento rápido para la continuidad en los proyectos de usuario en un único dispositivo. Es importante destacar que, para este prototipo, se ha priorizado esta solución frente a alternativas más robustas o escalables como bases de datos en tiempo real (ej. Firebase), las cuales se considerarían para fases de desarrollo futuras.

5.4. Retos Técnicos Abordados y Estrategias de Depuración

El desarrollo de un sistema híbrido de esta complejidad, integrando una librería de lógica de bloques con el entorno 3D y UI de Unity, ha presentado desafíos técnicos significativos.

- **Sincronización de Modelos y Vistas en MVC con Flotantes y Corrutinas:** El reto de mantener el `BlockModel` lógico (con posiciones `Vector2` abstractas) perfectamente sincronizado con el `RectTransform` visual en el Unity UI Canvas ha requerido una depuración exhaustiva.

Problemas de desajustes visuales o saltos en la posición de los bloques (especialmente al soltarlos tras un arrastre o al apilarse), así como errores de estado (‘BlockView’ o ‘ConnectionView’ mostrando un estado inconsistente con el modelo), han sido frecuentes. La complejidad aumenta con la naturaleza flotante de las coordenadas y la ejecución asíncrona de las animaciones del robot, donde la interfaz debe reflejar cambios precisos en momentos concretos.

■ **Bases de Datos de Conexiones en Memoria (BlockConnectionDB):**

Las BlockConnectionDBs de UBlockly utilizan estructuras de datos como cuadrantes o segmentos espaciales para optimizar la búsqueda de conexiones cercanas durante el arrastre. Asegurar que cada ConnectionModel estuviera correctamente registrado y desregistrado de estas bases de datos (‘ConnectionModel.Location’ debía estar perfectamente actualizada, y las funciones ‘ConnectionModel.AddConnection()’ y ‘ConnectionModel.RemoveConnection()’ sin errores de referencia a objetos ya destruidos) ha sido un punto crítico. La coherencia de estas estructuras internas, que son abstractas (en memoria) y no se ven directamente en la UI, es vital para que el sistema de snap funcione.

■ **Gestión de la Jerarquía de GameObjects y Disposición (Dispose/UnPlug):**

La correcta re-parentalización de GameObjects BlockView al conectarse y desconectarse (particularmente la lógica de bumping) y la eliminación de recursos ha requerido un cuidado extremo. Un error aquí podía llevar a fugas de memoria, bloques invisibles, bloques duplicados o zombies en la jerarquía, o que el robot saliera lanzado por una lógica de corrutinas mal coordinada que no espera la finalización de las animaciones.

■ **Estrategias de Depuración para Sistemas Complejos y Asíncronos:** Dada la complejidad y la interacción entre múltiples componentes MVC, corrutinas y la interfaz de Unity, las herramientas de depuración han sido esenciales:

- **Logueo Exhaustivo y Contextualizado (Logger):** Se ha implementado y utilizado de forma intensiva la clase `Logger`, una utilidad global para la emisión de mensajes ‘`Debug.Log`’, ‘`Debug.LogWarning`’ y ‘`Debug.LogError`’. *Decenas de puntos críticos en el código incluyen ‘Debug.Log’s detallados (a menudo comentados para evitar el ruido en la ejecución normal) que rastrean el estado exacto de variables, la entrada y salida de funciones, el estado de las corrutinas, y la pertenencia de objetos a bases de*

datos. Estos logs han sido el principal mecanismo para seguir el flujo de control, identificar desajustes y diagnosticar problemas de temporización.

- **Herramientas Visuales de Depuración en Unity:** El editor de Unity ha permitido la inspección en tiempo real de la jerarquía de `GameObjects`, los valores de los `RectTransforms`, el estado de los componentes `BlockView` y `ConnectionView`. Las herramientas de **Debugging de la ConnectionDB** (`ExportConnectionDBsButton` y `WorkspaceController.ExportConnectionDBState()`), desarrolladas para el prototipo, han sido fundamentales para *visualizar y exportar a JSON* el estado de las bases de datos de conexiones en memoria, permitiendo un análisis detallado de su coherencia. Esto ha sido invaluable para la depuración de la lógica de arrastre y snap.

La combinación de estas estrategias ha sido fundamental para navegar por la complejidad del desarrollo, asegurar la estabilidad del prototipo y justificar las decisiones de diseño a través de la evidencia empírica generada en el proceso de depuración.

6. Trabajos relacionados

Este apartado presenta un resumen comentado de los trabajos y proyectos más relevantes en el campo de la programación por bloques y la robótica educativa. El objetivo es contextualizar el prototipo desarrollado en este Trabajo Fin de Grado (TFG) dentro del panorama actual, identificando las tendencias, fortalezas y limitaciones de las soluciones existentes. Este análisis comparativo ha servido para fundamentar las decisiones de diseño e implementación del proyecto, buscando aportar valor y abordar necesidades específicas.

6.1. Plataformas de Programación por Bloques

En el ámbito de la programación visual, existen plataformas consolidadas que han servido como inspiración y punto de partida teórico para este proyecto.

- **Scratch** [22]: Desarrollado por el MIT Media Lab, es la referencia mundial para la enseñanza de la programación visual a niños y jóvenes. Su principal fortaleza reside en su interfaz intuitiva, el modelo de "drag and drop" de bloques, su categorización de colores, y su enfoque en la creación de narrativas y animaciones. Si bien es extremadamente accesible, su ecosistema es relativamente cerrado y no está diseñado para integraciones directas con entornos 3D o hardware robótico específico fuera de su plataforma.
- **Blockly** [14]: Librería de Google de código abierto que proporciona el framework base para construir editores de programación por bloques.

Destaca por su alta personalización y capacidad para generar código textual en múltiples lenguajes. Es la base técnica subyacente para numerosas soluciones, incluyendo extensiones hacia el aprendizaje de redes neuronales [23] o el control de dispositivos IoT [10]. Aunque potente, carece de la capa de experiencia de usuario y del ecosistema lúdico inherente a Scratch.

- **UBlockly [9]:** Como implementación de Blockly en C# para Unity, UBlockly ha sido la *base técnica* directa de este TFG. Su estructura modular y su capacidad para integrarse con el motor Unity (gestionando modelos de bloques, ejecución mediante corrutinas, y serialización) la convierten en una excelente plataforma para el desarrollo de un editor de bloques funcional y extensible. Nuestro proyecto se basa en la adaptación de UBlockly para replicar la experiencia de Scratch.

6.2. Simuladores y Entornos de Robótica Educativa

Para abordar la limitación de acceso a hardware físico, han surgido diversos simuladores que permiten a los estudiantes programar robots virtuales.

- **Simuladores genéricos basados en Unity:** Existen esfuerzos para desarrollar simuladores 3D de robótica en Unity para la enseñanza de programación o inteligencia artificial [15]. Estos proyectos, al igual que el presente TFG, aprovechan las potentes capacidades de renderizado 3D y el entorno de desarrollo versátil de Unity. Suelen enfocarse en una demostración de la viabilidad técnica o en contextos muy específicos (como la robótica industrial), pero a menudo carecen de una interfaz de programación por bloques totalmente integrada o de un fuerte enfoque en la educación preuniversitaria.
- **Plataformas web con simulación 3D:**
 - **Delightex (CoSpaces Edu):** Permite crear entornos 3D y aplicaciones de Realidad Virtual (VR)/Aumentada (AR) programables con bloques o JavaScript [4]. Es accesible vía navegador y ofrece un entorno visual atractivo para proyectos creativos, aunque la interacción con robots simulados es a menudo menos detallada que en motores dedicados.
 - **VEXcode VR y CoderZ:** Ofrecen entornos 3D para programar robots educativos específicos (VEX, GoPiGo) mediante bloques o

lenguajes textuales. Son excelentes para aprender el currículo de robótica asociado a esos kits, pero son plataformas con licencias o enfoque más específicos que restringen la flexibilidad o la personalización de robots y escenarios, y a menudo no están optimizadas para la creación de contenido 3D personalizado de forma sencilla [5].

■ **Simuladores para hardware específico:**

- **Robot Virtual Worlds :** Ofrece simuladores de alta fidelidad para robots LEGO EV3 o VEX. Proporcionan un alto grado de realismo físico y la capacidad de probar programas diseñados para robots específicos, siendo valiosos para entrenamientos detallados [16]. No obstante, suelen requerir mayores recursos computacionales y están fuertemente ligados a sus marcas, limitando su adaptabilidad.
- **Simuladores de Robótica Educativa de Makeblock:** Existen simuladores como Makeblock mBlock que integran un editor de bloques con entornos virtuales 2D/3D para los robots de su marca, promoviendo el pensamiento computacional [12]. Al igual que otros, están centrados en el ecosistema de la marca.

- **Juegos Serios y Entornos Inmersivos para Programación:** La investigación demuestra que los *serious games* y los entornos de realidad virtual/aumentada mejoran la motivación y el aprendizaje de la programación [1]. Ejemplos como **GeoBotsVR** combinan aprendizaje de robótica con un entorno inmersivo [18], mientras que VR-OCKS explora la programación visual 3D en Realidad Virtual para la resolución de puzzles de lógica de movimiento [17]. Estas soluciones son altamente atractivas, pero la complejidad de integrarlas con plataformas de código abierto para lograr extensibilidad o simulación de modelos robóticos específicos, y la accesibilidad en plataformas móviles de bajo coste, presenta retos aún no resueltos de forma óptima en el mercado generalista.

6.3. Herramientas para el Desarrollo de Interfaces de Bloques

Existen recursos y frameworks específicos diseñados para ayudar en la creación de interfaces de programación por bloques.

- **Blocks Engine 2** [13] Es un paquete de assets para Unity diseñado para facilitar el desarrollo de juegos y aplicaciones con interfaces de bloques. Este motor abstrae gran parte de la complejidad de la lógica de drag and drop, conexiones y validación de bloques en Unity. Representa una alternativa al enfoque de adaptar UBlockly. Si bien ofrece una solución más rápida lista para usar para la UI de bloques, mi decisión de trabajar con UBlockly se fundamenta en la necesidad de tener un control más granular sobre el *modelo lógico subyacente* de los bloques, su *motor de ejecución específico en C#*, y la flexibilidad para *personalizar profundamente* la lógica del sistema, lo cual es esencial para un Trabajo Fin de Grado enfocado en ingeniería.

6.4. Síntesis de la Contribución del Proyecto

A pesar de la existencia de numerosas herramientas y plataformas, la **integración holística de un sistema de programación visual completo, tipo Scratch, con una simulación 3D interactiva de un robot educativo concreto (LEGO WeDo 2.0) en un motor de juegos como Unity, optimizado para plataformas móviles accesibles**, sigue siendo un área con oportunidades significativas de desarrollo.

Pocas soluciones existentes logran el equilibrio óptimo entre:

- Una *experiencia de usuario lúdica e intuitiva* que motive a niños y adolescentes a programar.
- Un *motor de simulación robótica 3D visualmente convincente* y con la precisión necesaria para un aprendizaje eficaz.
- Una *arquitectura de software robusta y extensible* basada en un motor de juegos profesional como Unity y el lenguaje C#, permitiendo futuras evoluciones.
- Un *fuerte enfoque en la accesibilidad y el bajo coste*, reflejando un compromiso con los Objetivos de Desarrollo Sostenible.

La propuesta del presente TFG se distingue por abordar directamente este nicho. Al adaptar UBlockly y fusionar sus capacidades con las funcionalidades avanzadas de Unity, se busca crear un prototipo que no solo demuestra la viabilidad técnica de esta integración, sino que también establece las bases para un laboratorio virtual que vincule y emocione a los estudiantes, al tiempo que contribuye a la democratización de la educación STEAM.

7. Conclusiones y Líneas de trabajo futuras

Todo proyecto debe incluir las conclusiones que se derivan de su desarrollo, destacando los logros alcanzados y los aprendizajes obtenidos. Este capítulo final presenta una síntesis de los resultados del proyecto, tanto funcionales como técnicos. Además, se ofrece un informe crítico detallado que expone las limitaciones del prototipo actual y delinea una serie de líneas de trabajo futuras, que permiten trazar una hoja de ruta para la evolución y expansión del laboratorio virtual de programación y robótica.

7.1. Conclusiones

El presente Trabajo Fin de Grado ha abordado el desarrollo de un **prototipo funcional de laboratorio virtual de programación y robótica**, basado en el motor de videojuegos Unity 6 y el lenguaje de programación C#. A lo largo del proyecto, se ha implementado con éxito el **núcleo de un entorno de programación visual** (inspirado en Scratch) y un **sistema de simulación 3D básico** para el robot virtual LEGO WeDo 2.0. El proyecto ha demostrado la viabilidad técnica de combinar una interfaz de programación por bloques con una simulación robótica interactiva en un entorno accesible.

Los principales logros y aprendizajes del proyecto incluyen:

- **Implementación de un motor de programación visual viable:**
Se ha replicado con éxito la lógica de interacción visual de Scratch

(arrastré, conexión, disposición de categorías) y se ha establecido un robusto motor de ejecución directa de bloques, basado en la adaptación de UBlockly. Este motor ha demostrado la capacidad de interpretar y ejecutar una secuencia lineal de bloques (`motion_movesteps`, `event_whenflagclicked`) y de gestionar corrutinas para la animación fluida del robot.

- **Diseño de una arquitectura MVC efectiva:** La adopción del patrón Modelo-Vista-Controlador ha permitido una clara separación de intereses entre la lógica del programa (modelos de UBlockly como `BlockModel` y `ConnectionModel`), la representación visual (`BlockView` en Unity UI), y el control de interacciones (`BlockDragController`, `BlockConnectionController`). Esto garantiza la mantenibilidad y futura escalabilidad del sistema.
- **Integración exitosa de componentes:** Se ha logrado una integración fluida de los elementos de UBlockly (modelo de bloques, sistema de ejecución, serialización XML) con el entorno de Unity (UI Canvas, objetos 3D, corrutinas). Este proceso ha implicado resolver desafíos complejos relacionados con la sincronización de estados y posiciones entre los modelos lógicos y las vistas visuales de Unity.
- **Validación de un flujo completo de simulación:** Se ha verificado que un programa básico de bloques (ej. al presionar bandera verde, mover 10 pasos) se traduce correctamente en una acción visible y controlada por el robot virtual en el entorno 3D, demostrando la validez del concepto central del laboratorio.
- **Gestión de proyectos en un entorno Agile:** La aplicación de la metodología SCRUM, utilizando Zube y GitHub, ha sido fundamental para gestionar la complejidad, priorizar tareas y adaptarse a los retos técnicos emergentes durante el desarrollo del prototipo.

7.2. Líneas de Trabajo Futuras

El presente proyecto establece una base sólida sobre la que se pueden desarrollar mejoras y ampliaciones significativas para que el laboratorio virtual evolucione hacia una solución educativa integral. A continuación, se presentan algunas de las posibles líneas de trabajo futuras:

Extensión y Diversificación del Sistema de Bloques

Inicialmente, el prototipo implementa un conjunto limitado de bloques para demostrar su viabilidad. Para una solución completa, se plantea:

- **Ampliación de categorías de bloques:** Incorporar todas las categorías estándar de Scratch (ej. Sonido, Apariencia, Sensores).
- **Implementación de bloques avanzados:** Extender la funcionalidad para incluir estructuras de control más complejas como bucles, condicionales, funciones (procedimientos y bloques My Blocks), listas y operaciones con variables y operadores lógicos/matemáticos avanzados. Esto implicaría la adaptación de los intérpretes (`Cmdtors`) y la generación de prefabs visuales para cada nuevo tipo de bloque.
- **Expansión del modelo de eventos:** Desarrollar la capacidad para gestionar y responder a un rango más amplio de eventos, más allá de la bandera verde, permitiendo interacciones más complejas con el entorno virtual.
- **Sistema de depuración visual interactiva:** Crear herramientas visuales en la interfaz que permitan a los estudiantes seguir el flujo de ejecución del programa paso a paso, visualizar el estado de las variables y detectar errores en tiempo real, facilitando la comprensión y la resolución de problemas de lógica.
- **Integración de Bloques con Hardware Simulado Detallado:** Extender los bloques para representar el comportamiento detallado de motores (velocidad, dirección) y sensores (distancia, color, toque) del LEGO WeDo 2.0.

Mejora de la Simulación 3D e Interactividad

Actualmente, la simulación se centra en el movimiento básico del robot. Para mejorar la experiencia de aprendizaje y la inmersión, se pueden introducir mejoras como:

- **Entornos dinámicos e interactivos:** Diseñar escenarios 3D donde el robot pueda interactuar con obstáculos, objetos móviles o elementos activables (puertas, palancas), y responder a estímulos del entorno simulado (ej. zonas con distinto tipo de terreno que afecten el movimiento, etc.).

- **Simulación de física avanzada:** Integrar el motor de físicas de Unity (*Rigidbody*, *Collider*) de forma más compleja para dotar al robot y a los objetos del entorno de un comportamiento físico más realista (ej. colisiones, gravedad, rozamiento), lo que afectaría de manera auténtica la movilidad del robot y sus interacciones.
- **Soporte para múltiples modelos de robots:** Expandir el sistema para incluir y permitir la programación de distintos modelos de robots educativos (además del LEGO WeDo 2.0 básico), como LEGO SPIKE Essential, LEGO SPIKE Prime o LEGO MINDSTORMS EV3, cada uno con sus características y modelos 3D detallados.
- **Realidad Virtual (VR) y Aumentada (AR):** Explorar la portabilidad y adaptación del prototipo a plataformas de Realidad Virtual (VR) o Aumentada (AR), utilizando las capacidades nativas de Unity para ofrecer experiencias aún más inmersivas y atractivas para el aprendizaje.

Evaluación de la Efectividad Educativa y Usabilidad

Una limitación inherente a la fase de prototipado es la ausencia de una validación exhaustiva con usuarios reales. Para futuras adaptaciones, es fundamental realizar:

- **Estudios de usabilidad y experiencia de usuario (UX):** Realizar pruebas formales con grupos de estudiantes de diversas edades y niveles de experiencia, empleando metodologías como el test A/B o encuestas de satisfacción, para evaluar la facilidad de uso de la interfaz, la intuición del sistema de bloques y la efectividad general de la plataforma.
- **Análisis del impacto en el aprendizaje:** Diseñar y llevar a cabo estudios longitudinales para medir cómo la plataforma mejora las habilidades de programación y el pensamiento computacional de los estudiantes, en comparación con métodos de enseñanza tradicionales o con otras herramientas existentes.
- **Recopilación de métricas de interacción y comportamiento:** Implementar herramientas de análisis y telemetría para recolectar datos anónimos sobre cómo los estudiantes interactúan con el sistema (tiempo dedicado, bloques utilizados, errores comunes, caminos de solución), lo cual permitiría identificar áreas de mejora y optimizar la experiencia de aprendizaje.

Integración con Hardware Físico

Aunque el prototipo actual se centra en la simulación virtual, el objetivo a largo plazo incluye la interacción con robots reales. Las posibles mejoras en esta línea abarcan:

- **Conexión en tiempo real con LEGO WeDo 2.0 (Física):** Desarrollar un módulo de comunicación (mediante Bluetooth Low Energy) que permita la ejecución directa del código programado con bloques en el robot físico LEGO WeDo 2.0, creando un puente entre la programación virtual y la experiencia tangible.
- **Compatibilidad con otros kits de robótica:** Ampliar el sistema para permitir la programación y el control de otros dispositivos de robótica educativa, como Arduino o micro:bit, a través de interfaces de programación de bajo nivel (APIs o protocolos de comunicación específicos) para esos dispositivos.

Optimización del Rendimiento, Portabilidad y Localización

Para maximizar la accesibilidad y el alcance del laboratorio virtual, es esencial abordar aspectos de optimización y portabilidad.

- **Optimización del rendimiento en Unity:** Llevar a cabo una fase de optimización exhaustiva para reducir el consumo de recursos de la aplicación (CPU, GPU, RAM), lo cual permitiría su ejecución más fluida en dispositivos de bajo rendimiento, como tablets básicas o computadoras más antiguas, maximizando la inclusión.
- **Portabilidad a Versión Web del Entorno:** Desarrollar una versión ejecutable del laboratorio virtual directamente en navegadores web (utilizando tecnologías como Unity WebGL), eliminando la necesidad de instalación y facilitando un acceso instantáneo desde cualquier dispositivo con conexión a internet.
- **Localización del Sistema:** Traducir la interfaz de usuario, las descripciones de los bloques y los tutoriales a múltiples idiomas. Esto expandiría el alcance educativo de la plataforma a nivel global, eliminando barreras lingüísticas.

7.3. Conclusión Final

El desarrollo de este laboratorio virtual representa un avance significativo en la enseñanza de la programación y la robótica a través de entornos visuales y simulados. Mediante la adaptación de UBlockly en Unity, se ha logrado un prototipo funcional que demuestra la viabilidad técnica y el potencial educativo de esta innovadora herramienta. La aplicación de los conocimientos de ingeniería informática en este proyecto no solo ha permitido construir un sistema capaz de vincular y emocionar a los estudiantes, transformando la compleja tarea de programar en una actividad lúdica y accesible.

Las futuras líneas de trabajo esbozadas demuestran el inmenso potencial de evolución del prototipo. Desde la expansión del repertorio de bloques y la mejora del realismo de la simulación, hasta la integración con hardware físico y la optimización de la accesibilidad global, cada una de estas áreas abre caminos para convertir este prototipo en una herramienta educativa más robusta, inclusiva y universal. Este trabajo no solo es una prueba de mis habilidades como ingeniero, sino también un compromiso con la democratización del conocimiento y el uso de la tecnología como motor de un futuro más justo y equitativo.

Bibliografía

- [1] Fahad Alshahrani, Areej Aldhahri, Abeer Aldhahri, and Zahid Ali. Gamified learning of block-based programming with artificial intelligence concepts for k-12 students: an experimental study. *Smart Learning Environments*, 11(1):33, 2024. [En línea; consultado el 29-mayo-2025].
- [2] Jesús R. Campaña, Ana E. Marín, María Ros, Daniel Sánchez, Juan M. Medina, María Amparo Vila, María Dolores Ruiz, Manuel P. Cuéllar, and María J. Martín-Bautista. Metodologías activas y gamificación en las asignaturas de iniciación a la programación. In *Actas de las JENUI - Vol. 1 (2016)*, Almería, jul 2016. [En línea; consultado el 12-febrero-2025].
- [3] Code Week EU. The importance of early coding education. <https://codeweek.eu/blog/the-importance-of-early-coding-education/>, 2020. [En línea; consultado el 29-mayo-2025].
- [4] Codespaces. Delightex is a company working towards offering innovative technologies that contribute to transforming education. <https://www.delightex.com/>, 2025. [En línea; consultado el 30-mayo-2025].
- [5] Rafael Arnay Coromoto León Cristian Manuel Ángel Díaz, Eduardo Segredo. Simulador de robótica educativa para la promoción del pensamiento computacional. 2020. [En línea; consultado el 28-enero-2025].
- [6] María José L. Domínguez, Juan R. Arévalo, David M. Pérez, Ignacio L. Durán, José P. D. López, and Manuel J. Gil. Motivation and engagement of students: A case study of automatics and robotics projects. *Sensors*, 24(10):384850438.

- [7] Brandon Fuchs. Incorporating robotic technologies into stem curricula through scratch robotics. Master's thesis, Concordia University, Saint Paul, 2021. [En línea; consultado el 29-mayo-2025].
- [8] Github. Github copilot - ai that buidls with you. <https://github.com/features/copilot>, 2025. [En línea, consultado el 30-mayo-2025].
- [9] Imagicbell. UBlockly - Introducción a la Programación con Bloques en Unity, 2021. [En línea; consultado el 20-febrero-2025].
- [10] Florin Cătălin Ionescu and Radu V. Vasiu. Developing a programming-learning platform using block-based approach and iot device for students. In *ICERD 2020: EAI International Conference on Electronic Communications, Internet of Things and Cloud Computing*, pages 102–108. Springer, 2020. [En línea; consultado el 29-mayo-2025].
- [11] David Klass. The benefits of teaching kids to code. <https://www.bc.edu/bc-web/sites/bc-magazine/fall-2023-issue/linden-lane/the-benefits-of-teaching-kids-to-code.html>, 2023. [En línea; consultado el 29-mayo-2025].
- [12] MakeBlock. Makeblock software center. <https://www.makeblock.com/pages/software/>, 2025. [En línea, consultado el 30-mayo-2025].
- [13] MeadowGames. Blocks engine 2. <https://meadowgames.com/>, 2025. [En línea, consultado el 30-mayo-2025].
- [14] Roberto Muñoz, Thiago S. Barcelos, Rodolfo Villarroel, Marta Barriá, Carlos Becerra, Rene Noel, and Ismar Frango Silveira. Uso de scratch y lego mindstorms como apoyo a la docencia en fundamentos de programación. In *Actas de las XXI Jornadas de Enseñanza Universitaria de la Informática, JENUI 2015*, Andorra La Vella, jul 2015. [En línea; consultado el 12-febrero-2025].
- [15] Héctor Pérez, José L. García, and Marta Fernández. Simulador 3d de robótica distribuido en unity para enseñanza de programación e inteligencia artificial. *ResearchGate*, 2024. [En línea; consultado el 20-febrero-2025].
- [16] Robomatter, Inc. Robot virtual worlds. <https://www.robotvirtualworlds.com//>, 2025. [En línea, consultado el 30-mayo-2025].

- [17] Rafael J. Segura, Francisco J. del Pino, Carlos J. Ogáyar, and Antonia J. Rueda. Vr-ocks: A virtual reality game for learning the basic concepts of programming. *Computer Applications in Engineering Education*, 28(1):31–41, 2020.
- [18] John Smith and Jane Doe. Geobotsvr: A robotics learning game for beginners with hands-on learning simulation. *arXiv*, 2024. [En línea; consultado el 20-febrero-2025].
- [19] Juan P. Torres, Miguel A. Castro, María F. Flores, José I. Villagran, Pamela F. Muñoz, and Claudia I. Contreras. A methodological approach to the teaching stem skills in latin america through educational robotics for school teachers. *Educ. Sci.*, 11(1):25, 2021. [En línea; consultado el 29-mayo-2025].
- [20] Naciones Unidas. Educación, objetivos de desarrollo sostenible, 2023. [En línea; consultado el 20-febrero-2025].
- [21] Maximiliano Paredes Velasco and J. Ángel Velázquez-Iturbide. Una asignatura para la formación del profesorado en programación mediante lenguajes basados en bloques. In *Actas de las JENUI - Vol. 7 (2022)*, pages 343–350, La Coruña, jul 2022. [En línea; consultado el 11-febrero-2025].
- [22] Maximiliano Paredes Velasco, Jesús Ángel Velázquez Iturbide, Sergio Caveró Díaz, and Daniel Palacios Alonso. Mejora de una asignatura para la formación del profesorado en programación basada en bloques. In *Actas de las JENUI - Vol. 8 (2023)*, pages 269–276, Granada, jul 2023. [En línea; consultado el 11-febrero-2025].
- [23] Lei Wang, Xiangang Zhou, Zhiyong Wang, Qingsong Zang, and Chunqing Tan. Towards a block-based programming language and its web-based environment to teach deep learning. *Frontiers in Education*, 6:657895, 2021. [En línea; consultado el 29-mayo-2025].
- [24] Zube Inc. Zube - project development managemet for agile development. <https://zube.io>, 2025.